

HW 05 — Adaptive Runge-Kutta ODE Solver and Baseball Trajectories

Overview

This assignment implements an adaptive Runge-Kutta ODE integrator with step-doubling error control and Richardson extrapolation, then applies it to simulate the trajectories of a baseball curveball and fastball including drag and Magnus force.

Part 1: Adaptive Runge-Kutta Solver

Problem Statement

Implement `rk_adaptive(f, t0, y0, t_end, tol, order)` that integrates the ODE $\mathbf{y}' = f(t, \mathbf{y})$ from t_0 to t_{end} with local error controlled to `tol`. Support both RK3 and RK4. Validate on $y' = -y$ (exact: e^{-t}) and a sum-of-Gaussians benchmark. Then validate on the baseball problem.

The Runge-Kutta Step

A single Runge-Kutta step of order p advances $\mathbf{y}$ from t to $t + h$ using a weighted combination of derivative evaluations (stages):

RK3 (Kutta's third-order method):

```
k1 = f(t, y)
k2 = f(t + h/2, y + (h/2)k1)
k3 = f(t + h, y - h·k1 + 2h·k2)
y_new = y + (h/6)(k1 + 4k2 + k3)
```

3 function evaluations. Local truncation error $O(h^4)$, global error $O(h^3)$.

RK4 (the classical 4th-order Runge-Kutta):

```
k1 = f(t, y)
k2 = f(t + h/2, y + (h/2)k1)
k3 = f(t + h/2, y + (h/2)k2)
k4 = f(t + h, y + h·k3)
y_new = y + (h/6)(k1 + 2k2 + 2k3 + k4)
```

4 function evaluations. Local truncation error $O(h^5)$, global error $O(h^4)$.

The weights come from the Butcher tableau, which ensures that the weighted sum matches the Taylor expansion of $\mathbf{y}(t + h)$ up to the specified order.

Step-Doubling Error Control

To estimate the local error without knowing the exact solution, the method takes: 1. One step of size h : gives $\mathbf{y}_{\text{full}}$. 2. Two steps of size $h/2$: gives $\mathbf{y}_{\text{half}}$.

The difference $\|\mathbf{y}_{\text{half}} - \mathbf{y}_{\text{full}}\|_{\infty}$ estimates the local truncation error (it scales as h^{p+1} for order p).

- If `err < tol`: accept the step, advance t , and try to increase h .
- If `err > tol`: reject the step, shrink h , and retry.

Step size adjustment formula (based on the $p + 1$ error scaling):

$$h_{\text{new}} = h \cdot \min \left(2, 0.9 \cdot \left(\frac{\text{tol}}{\text{err}} \right)^{1/(p+1)} \right)$$

The factor 0.9 is a safety margin; the factor 2 prevents the step from growing too fast.

Richardson Extrapolation

After accepting a step, rather than returning $\mathbf{y}_{\text{half}}$ (which has error $O(h^{p+1})$), the code applies Richardson extrapolation to get a higher-order estimate:

$$\mathbf{y}_{\text{best}} = \mathbf{y}_{\text{half}} + \frac{\mathbf{y}_{\text{half}} - \mathbf{y}_{\text{full}}}{2^p - 1}$$

This cancels the leading error term, giving an answer accurate to $O(h^{p+2})$ instead of $O(h^{p+1})$. For RK3 ($p = 3$), the denominator is $2^3 - 1 = 7$; for RK4 ($p = 4$), it is $2^4 - 1 = 15$.

Implementation:

```
scale = 2**order - 1
...
y_full, _ = rk_step(f, t, y, h, order)
y_mid, _ = rk_step(f, t, y, h/2, order)
y_half, _ = rk_step(f, t + h/2, y_mid, h/2, order)
err = np.max(np.abs(y_half - y_full))

if err <= tol:
    y = y_half + (y_half - y_full) / scale # Richardson extrapolation
    t += h
    h *= min(2.0, 0.9 * (tol / err)**(1.0 / (order + 1)))
else:
    h *= max(0.1, 0.9 * (tol / err)**(1.0 / (order + 1)))
```

Validation Results

Exponential decay $y' = -y$, $y(0) = 1$, **exact** $y(5) = e^{-5}$:

Order	tol	Error	Steps
RK3	10^{-3}	2.2×10^{-5}	12
RK3	10^{-9}	4.1×10^{-11}	249
RK4	10^{-3}	8.5×10^{-6}	10
RK4	10^{-9}	1.6×10^{-11}	86

RK4 reaches the same accuracy as RK3 with fewer steps, consistent with its higher order.

Sum-of-Gaussians benchmark: Five narrow Gaussians (width $a = 0.1$) centered at $x = -5, -1, 1, 4, 7$; exact integral is 5. The adaptive solver concentrates steps near each Gaussian and takes large steps in between, achieving high accuracy efficiently.

Part 2: Baseball Trajectory

Problem Statement

Simulate a baseball thrown from a pitcher to a batter (distance 60 ft = 18.29 m) subject to:
1. **Gravity** $-g\hat{z}$ 2. **Aerodynamic drag** (velocity-dependent) 3. **Magnus force** (from spin)

Compare a curveball ($v_0 = 85 \text{ mph} = 38.00 \text{ m/s}$, $\varphi = 45^\circ$) and a fastball ($v_0 = 95 \text{ mph} = 42.47 \text{ m/s}$, $\varphi = 225^\circ$) both with spin $\omega = 1800 \text{ rpm}$.

Equations of Motion

State vector: $\mathbf{y} = [x, y, z, v_x, v_y, v_z]$ (position + velocity, SI units).

The aerodynamic drag coefficient is velocity-dependent:

$$F(v) = 0.0039 + \frac{0.0058}{1 + e^{(v-35)/5}}$$

This is a smooth sigmoidal model: at low speeds the drag is dominated by the constant term (≈ 0.004); at high speeds the second term decreases the drag coefficient.

The equations of motion:

$$\dot{v}_x = -F(v) v v_x + B\omega(v_z \sin \varphi - v_y \cos \varphi)$$

$$\dot{v}_y = -F(v) v v_y + B\omega v_x \cos \varphi$$

$$\dot{v}_z = -g - F(v) v v_z - B\omega v_x \sin \varphi$$

where $B = 4.1 \times 10^{-4} \text{ m}^{-1}$ is the Magnus coefficient and φ is the spin axis orientation. The drag terms $-F(v) v v_i$ are opposite to each velocity component; the Magnus (cross-product) terms depend on the spin axis direction φ .

Initial conditions: Ball starts at origin with $v_x = v_0 \cos \theta$, $v_z = v_0 \sin \theta$ (small upward angle $\theta = 1^\circ$), $v_y = 0$.

Integration time: $t_f = L/v_0$ where $L = 18.288 \text{ m}$ (simplified — ignores slowing down, but the problem specifies this).

Implementation

```
def baseball_rhs(t, state, omega, phi):
    _, _, _, vx, vy, vz = state
    v = np.sqrt(vx**2 + vy**2 + vz**2)
    Fv = 0.0039 + 0.0058 / (1.0 + np.exp((v - 35.0) / 5.0))
    ax = -Fv*v*vx + B*omega*(vz*np.sin(phi) - vy*np.cos(phi))
    ay = -Fv*v*vy + B*omega*vx*np.cos(phi)
    az = -g - Fv*v*vz - B*omega*vx*np.sin(phi)
    return np.array([vx, vy, vz, ax, ay, az])
```

Results

Curveball ($\varphi = 45^\circ$): deflects $y \approx +0.22$ m sideways and drops $z \approx -1.01$ m. The large drop shows how gravity and downward Magnus combine.

Fastball ($\varphi = 225^\circ$, opposite spin axis): deflects $y \approx -0.20$ m (opposite direction) and drops only $z \approx -0.37$ m. The smaller drop is partly because the faster ball spends less time in flight, and the Magnus force has an upward component that partially counters gravity.

Accuracy vs. cost comparison: Running the curveball at reduced tolerances (10^{-1} to 10^{-7}) with both RK3 and RK4:

- RK4 at `tol=1e-1` gives $|\Delta y| \approx 1.4 \times 10^{-11}$ m from the high-accuracy reference — effectively machine precision — with only 84 function calls.
- RK3 needs `tol=1e-7` (135 calls) to reach similar accuracy.

This demonstrates that for smooth ODEs like this one, RK4's higher order per evaluation makes it significantly more efficient than RK3.

Key Takeaways

- **Runge-Kutta methods** build a high-order step from several derivative evaluations (stages) whose weights are chosen to match the Taylor expansion. RK4 (4 stages, global $O(h^4)$) is the standard workhorse.
- **Step-doubling estimates the local error for free:** compare one step of size h against two of size $h/2$; their difference scales as h^{p+1} and serves as the error estimate driving step-size control.
- **Adaptive step-size control:** accept if `err < tol`, else shrink and retry; rescale by $(\text{tol}/\text{err})^{1/(p+1)}$ with a safety factor. The solver automatically takes small steps in fast regions and large steps in smooth ones.
- **Richardson extrapolation** combines the two estimates, $y_{\text{best}} = y_{\text{half}} + \frac{y_{\text{half}} - y_{\text{full}}}{2^p - 1}$, cancelling the leading error term to gain a free order of accuracy.
- **Higher order is more efficient for smooth problems:** RK4 reaches a target accuracy in far fewer steps/calls than RK3, even though each step costs more.
- **Reduce any ODE to first order** by stacking into a state vector (here $[x, y, z, v_x, v_y, v_z]$); the same scalar RK machinery then integrates the whole system.